



.NET Fundamental

Parham Darvishi

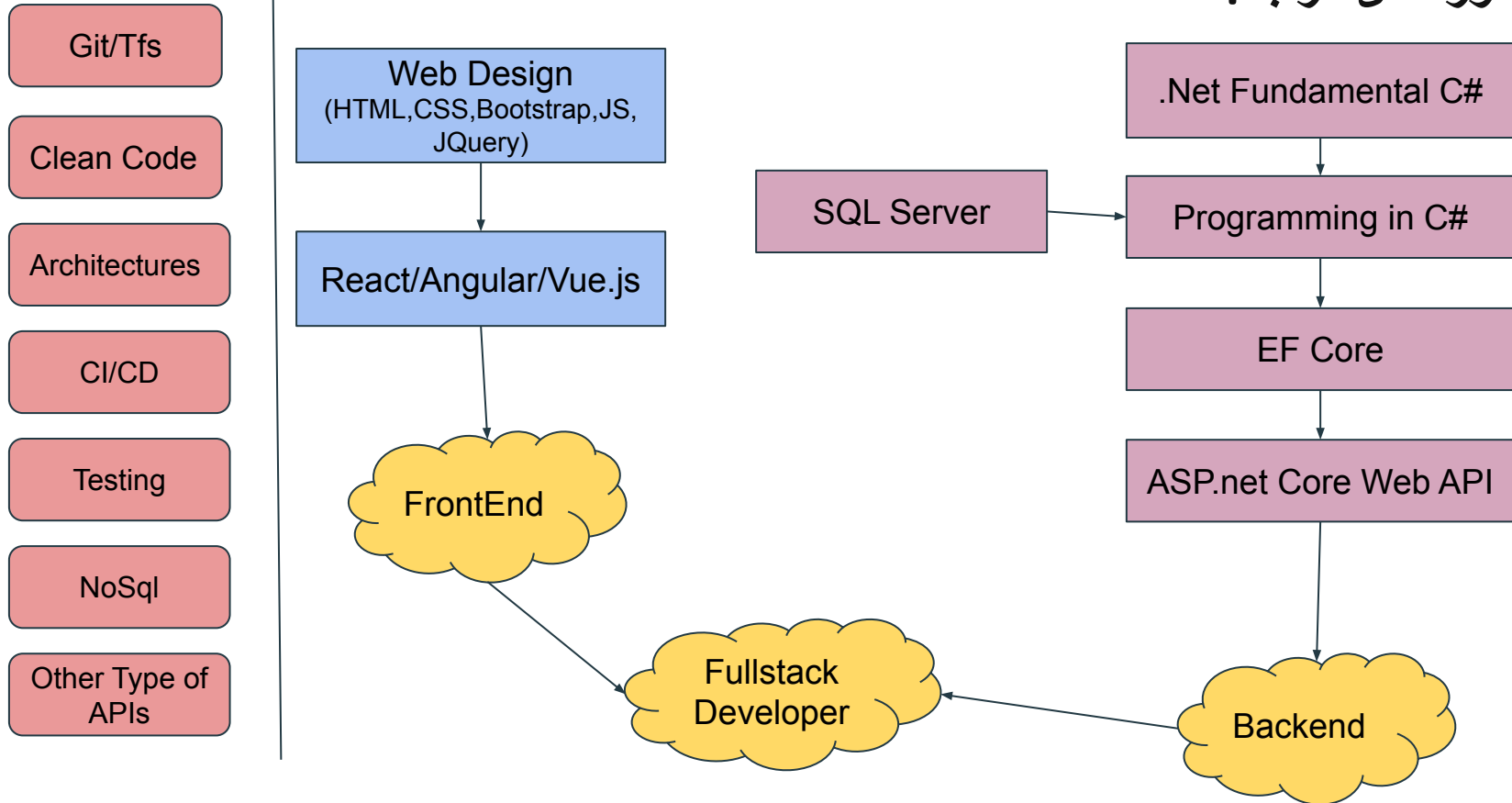
Linkedin: <https://www.linkedin.com/in/parham-darvishi-26b6b8161/>

Github: <https://github.com/parhamdarvishi>

Phone Number: +989129564205

Email: darvishiparham14@gmail.com

دوره‌های مرتبط:



پیش نیازها:

الگوریتم چیست؟

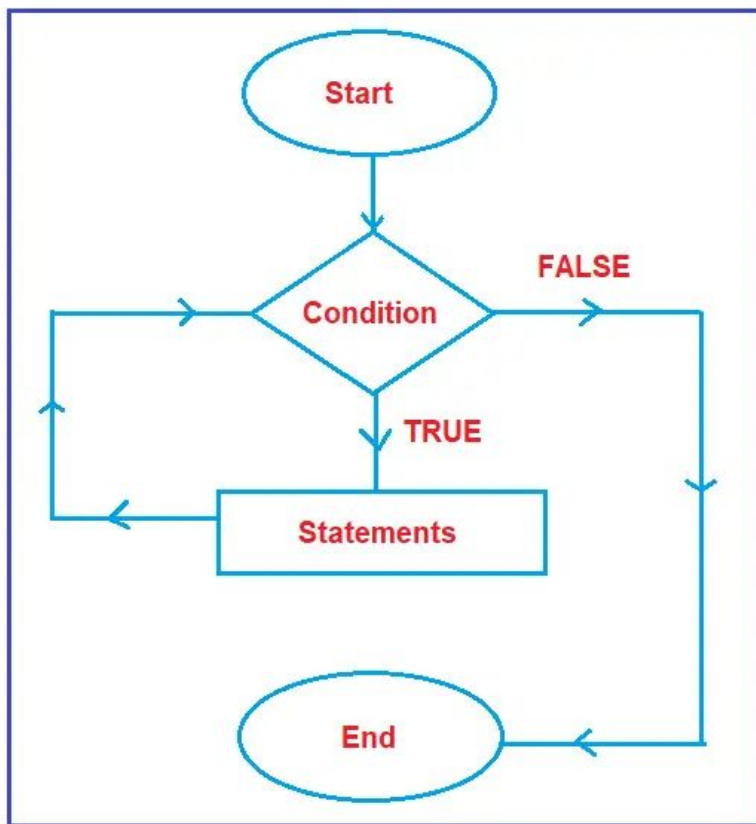
یک دنباله محدود از دستورالعمل‌های کاملاً تعریف شده است.

فلوچارت چیست؟

در واقع نقشه‌ای است که برنامه‌نویسان قبل از نوشتن برنامه به زبان برنامه‌نویسی اصلی آن را ترسیم می‌کنند. با مروری بر فلوچارت، روند اجرای عملیات، مراحل و جزئیات برنامه و ورودی و خروجی هر مرحله از برنامه مشخص می‌شود.

اگر شما اطلاعات بهتری نسبت به CPU & Memory & Application داشته باشید قطعاً مطالب رو بهتر درک خواهید کرد. (مشاهده اسلایدهای مربوط به این مبحث)

نمونه ای از فلوچارت:



تعریفون از زبان برنامه نویسی چیه؟

A programming language is any set of **rules** that convert **strings** to various kinds of **machine code output**.

حال فرض کنید زبان برنامه نویسی خودتون رو نوشتید آیا قبول دارید نیاز به نرم افزار **ی** دارید که بتواند این فرآیند تبدیل به machine code را انجام دهد! به این نرم افزار **کامپایلر** میگویند.

اما واقعا نرم افزار چیست؟ بریم ببینیم..

انواع نرم افزار:

● نرم افزارهای سیستمی: میان سیستم عامل و سایر نرم افزارهای کاربردی قرار میگیرد.

○ سیستم عامل

○ اسمبلرها

○ کامپایلرها

● نرم افزارهای کاربردی:

○ مرورگر

○ نرم افزارهای گرافیکی

فرض میکنم میخوام یک زبان برنامه نویسی خلق کنم!

میگم اول بیا یه مقدار زبان رو یاد بگیریم!

جمله "تو کی هستی؟" را تصور کن. من برای اینکه این جمله رو بفهمم چه مراحل رو طی کردم؟

آیا جمله "تو کی هستی؟" معنا دارد؟

آیا جمله "هستی کی تو؟" معنا دارد؟

پس ما ابتدا هر کلمه را جدا جدا درک میکنیم. (token) و سپس معنای کلی را بررسی میکند.

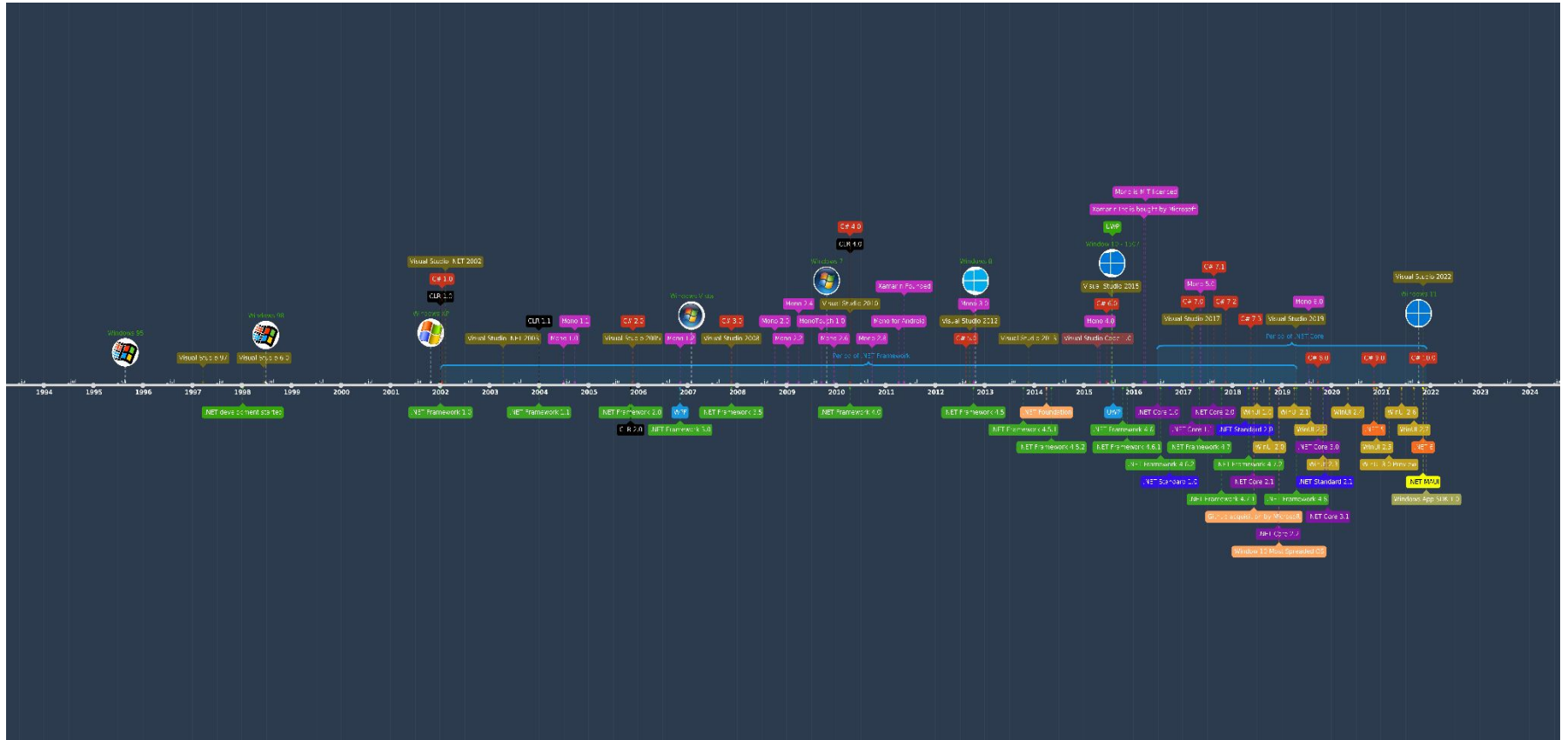
سوال: Net Framework چیست؟

فریمورک راه حل های از پیش نوشته شده و دسته بندی شده ای را ارائه می دهند که شما می توانید به کمک آنها ایرادات و خطاهای معمول برنامه نویسی را به حداقل برسانید.

یک framework در قالب Platform net است.

فریمورک با کتابخانه تفاوت دارد. (این جمله کلیدی را به خاطر داشته باش).

.Net History



C# Version

- **C# 1.0**
 - .NET Framework 1.0/1.1
 - Visual Studio .NET 2002
 - Features:
 - Basic features
- **C# 2.0**
 - .NET Framework 2.0
 - Visual Studio 2005
 - Features:
 - Generics
 - Partial types
 - Anonymous methods
 - Iterators
 - Nullable types
 - Private setters (properties)
 - Method group conversions (delegates)
 - Covariance and Contra-variance
 - Static classes

- **C# 3.0**

- .NET Framework 3.0\3.5
- Visual Studio 2008
- Features:
 - Implicitly typed local variables
 - Object and collection initializers
 - Auto-Implemented properties
 - Anonymous types
 - Extension methods
 - Query expressions
 - Lambda expressions
 - Expression trees
 - Partial Methods

- **C# 4.0**
 - .NET Framework 4.0
 - Visual Studio 2010
 - Features:
 - Dynamic binding (late binding)
 - Named and optional arguments
 - Generic co- and contravariance
 - Embedded interop type

- **C# 5.0**
 - .NET Framework 4.5
 - Visual Studio 2012/2013
 - Features:
 - Async features
 - Caller information

- **C# 6.0**
 - .NET Framework 4.6
 - Visual Studio 2013/2015
 - Features:
 - Expression Bodied Methods
 - Auto-property initializer
 - nameof Expression
 - Primary constructor
 - Await in catch block
 - Exception Filter
 - String Interpolation

- **C# 7.0**
 - .NET Core 2.0
 - Visual Studio 2017
 - Features:
 - out variables
 - Tuples
 - Discards
 - Pattern Matching
 - Local functions
 - Generalized async return types

- **C# 8.0**
 - .NET Core 3.0
 - Visual Studio 2019
 - Features:
 - Readonly members
 - Default interface methods
 - Using declarations
 - Static local functions
 - Disposable ref structs
 - Nullable reference types

- **C# 9.0**

- .NET 5.0
- Visual Studio 2019
- Features:
 - Records
 - Init-only properties
 - Top-level statements
 - Init accessors and readonly fields
 - With-expressions
 - Value-based equality

- **C# 10.0**

- .NET 6.0
- Visual Studio 2022
- Features:
 - Record structs
 - Global using directives
 - File-scoped namespace declaration
 - Extended Property Patterns
 - Null Parameter Checking
 - Constant interpolated strings

- **C# 11.0**

- .NET 7.0
- Visual Studio 2022
- Features:
 - Raw string literals
 - Generic math support
 - Generic attributes
 - UTF-8 string literals
 - Newlines in string interpolation expressions
 - List patterns
 - File-local types
 - Required members
 - Auto-default structs
 - Pattern match `Span<char>` on a constant string
 - Extended `nameof` scope
 - Numeric `IntPtr`
 - `ref` fields and `scoped ref`
 - Improved method group conversion to delegate

Architecture of .NET Framework

Base Class Library(BCL)

حاوی کلاس ها و اسمبلی های کاربردی از پیش تعریف شده که قابلیت استفاده آنها بر اساس Namespace های مختلف وجود دارد مانند:

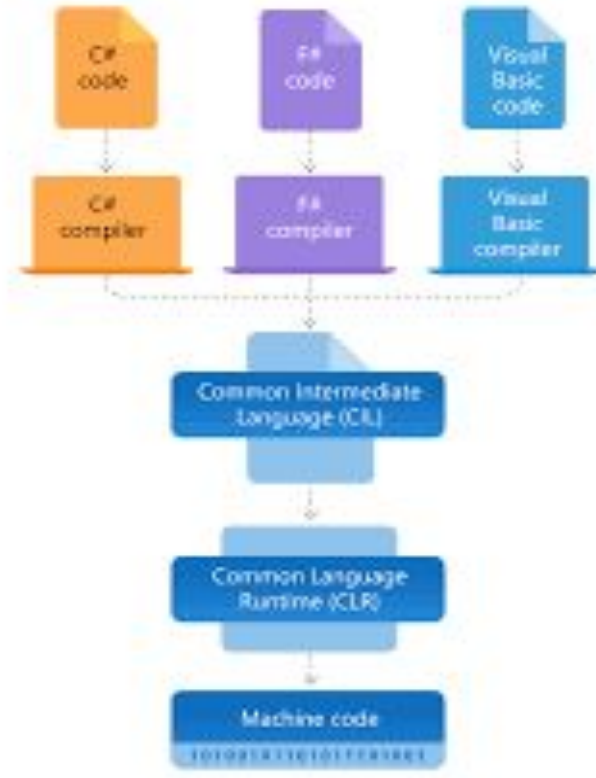
APIs for reading and writing files, connecting to databases, drawing,...

Common Language Runtime(CLR)

وظیفه اجرای برنامه ها را برعهده دارد مانند:

thread management, garbage collection, type-safety, exception handling,...

نحوه اجرا اپلیکیشنها:



C# (.cs file) -> C# compiler -> IL Code(exe file)

IL code(exe file) run inside CLR -> JIT Compiler -> native code

فارغ از هر زبانی اگر بر بستر Net. باشد به IL تبدیل میشود.
میتوان گفت CLR مثل یک virtual machine برای دات نت پلتفرم است.
میتوان گفت JIT در داخل CLR است.

سناریو اجرا در قدیم و حال:

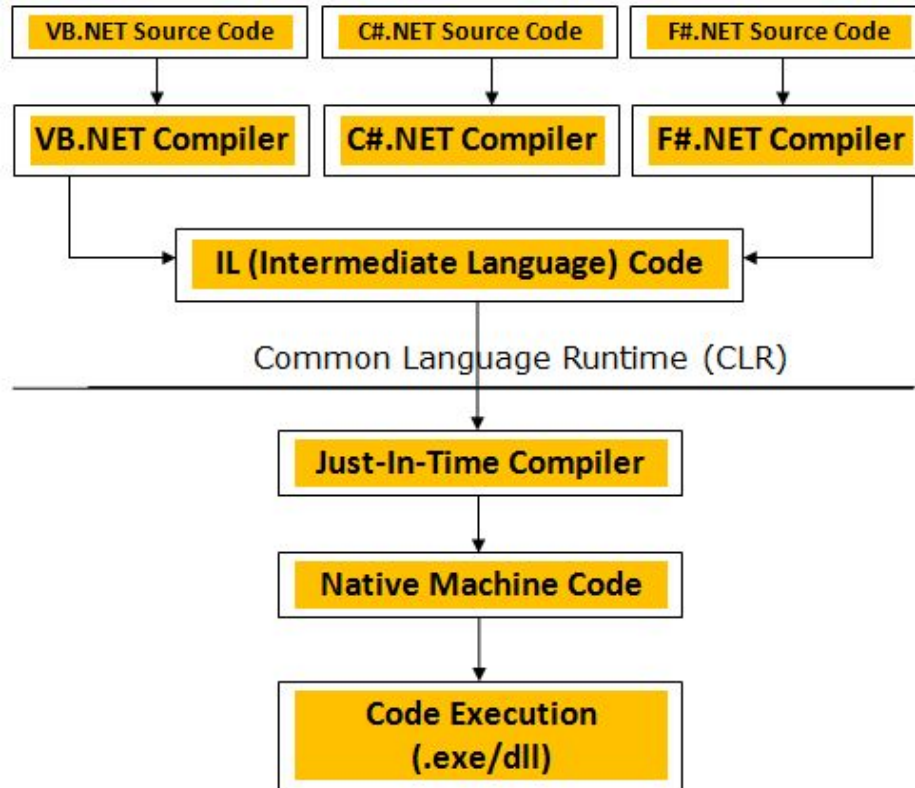
قدیم اگر ما با زبانی مثل C کدی می نوشتیم و این کد را با سیستمی با معماری/مشخصات X خروجی می گرفتیم آنگاه native code یی برای آن ماشین تشکیل میشد.

حال اگر میخواستیم خروجی را در ماشین دیگری اجرا بگیریم چه اتفاقی می افتاد؟

در اینجا بود که ایده‌ای از زبان Java آمد و آن هم به جای تبدیل مستقیم به native code تبدیل به یک مفهوم مستقل از سخت افزار است.

پس ماشینی که قرار است کد ما را اجرا کند کافیسست که فقط CLR را داشته باشد.

CLR Architecture



زبان برنامه نویسی سی شارپ؟

Based on C and C++

Some Features more than C (Accessing and manipulating system memory)

modern, object-oriented, and type-safe programming language

Fully supported .Net Framework features

با سی شارپ چه برنامه‌هایی میتوان نوشت؟

Desktop Applications

- Creating Windows Forms Applications
- Creating Windows Presentation Foundation (WPF) Applications
- Creating Mobile and Embedded Applications

Windows Store Applications

- Create new type of applications for touch devices

Web Applications

- Creating ASP.Net Web Applications
- Web Forms
- MVC
- Silver Light

WCF Services

- Create for support distributed Applications (over local network and internet)

Other Types Creating

- Console Applications
- Creating Custom Class Libraries
- Creating Components

C# IDEs

IDE محیطی برای کد نویسی، تست، «اشکال زدایی» (Debugging) و سایر موارد این چنینی فراهم می کنند.

Visual Studio Code: open-source text editor and IDE developed by Microsoft

Features:

- Support for multi-platforms like macOS, Windows, and Linux.
- The IDE is highly customizable and extensible by importing extensions (themes, debuggers, languages, additional services, etc.) from the the Visual Studio Marketplace.

Visual Studio: for .NET ecosystem

Features:

- Visual Studio is used for building web applications, websites, mobile applications, Windows applications, and video games.

How to Run C# in VS code on windows 10

1. Install Vs Code (<https://code.visualstudio.com/download>)
2. Download Microsoft Dotnet Core SDK
3. From (<https://dotnet.microsoft.com/en-us/download>)
4. Set up downloaded file as Run as administrator
5. Open cmd and write "dotnet --version"
6. Install C# extension for VS code

How to Create C# Project in VS code on windows 10

1. Open a folder
2. Open cmd in that path
3. Open Vs Code in that path
4. Write "dotnet new console"
5. See the project
6. Write "dotnet run"
7. Another way to run your code is download "Code Runner" extension
8. To active "Code Runner" -> File -> Preferences -> Settings -> Search for "run in terminal" and Active it -> Go to Settings.Json -> Type "code-runner.executorMap"
9. Go to "csharp" and edit value with "cd \$dir; dotnet run \$fileName"

Intro Visual Studio

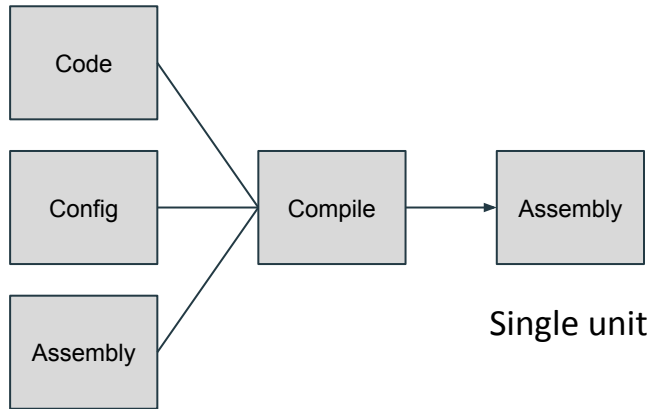
- The VS automates the steps required to compile
- The VS has powerful text editor
- VS includes designers for Windows Forms, Web Forms, and other applications
- VS includes several wizards that automate common tasks
- VS contains many powerful tools for visualization and navigation through elements of your projects
- You can create application, deployment projects easily
- VS enables you to use advanced debugging techniques

سوال: SDK چیست؟

ابزار «کیت توسعه نرم افزار» یا همان SDK که مخفف «Software Development Kit» است، مجموعه‌ای از ابزارها و برنامه‌های نرم‌افزاری به حساب می‌آید که توسط فروشندگان و سازندگان سخت افزار و نرم افزار ارائه می‌شوند. همچنین، توسعه‌دهندگان می‌توانند از آن برای ساخت برنامه‌ها و اپلیکیشن‌های پلتفرم‌های خاص استفاده کنند. ارائه دهنندگان SDK امکانات مورد نیاز را در دسترس توسعه‌دهندگان قرار می‌دهند تا آن‌ها این قابلیت‌ها را به راحتی در برنامه‌هایشان به کار بگیرند.

Assembly:

- **Microsoft definition:** fundamental unit of deployment
- **My definition:** its precompiled chunk of .Net Code which can run by CLR



علت اینکه شما یک فایل اجرایی دارید همین مفهوم اسمبلی است.

انواع اسمبلی:

- DLL از واژه Dynamic Link Library می آید. (نقطه آغازین ندارد و Memory Address نمی گیرد و میتواند در داخل جاهای دیگر استفاده شود).
- EXE از واژه Executable می آید. (نقطه آغازین دارد و Memory Address میگیرد).

Visual Studio 2022 Development Env

پنجره‌های موجود:

- Toolbox Window ●
- Solution Explorer ●
- Team Explorer ●
- Properties Window ●
- Bottom Window ●

بررسی مختصر منوها و نوار ابزار: File, View, Edit, Tools, ...

compiler-generated folders

- The obj folder contains one compiled object file for each source code file. These objects haven't been linked together into a final executable yet.
- The bin folder contains the binary executable for the application or class library. We will look at this in more detail in Chapter 7, Packaging and Distributing .NET Types.

Understanding top-level programs

This is because some code is written for you by the compiler when you target .NET 6 or later.

If you had created the project with .NET SDK 5.0 or earlier, or if you had selected the check box labeled Do not use top-level statements

- There can be only one file like this in a project.
- Any using statements must go at the top of the file.
- If you declare any classes or other types, they must go at the bottom of the file.
- Although you should name the method Main if you explicitly define it, the method is named <Main>\$ when created by the compiler.

Console Applications

- برنامه ای برای دریافت اطلاعات و نمایش خروجی در مد Line Command
- دریافت ورودی هایی از جنس Character و نمایش آنها
- برنامه ای بدون داشتن ظاهر گرافیکی خاص
- قابلیت اجرای ساده مخصوصا با استفاده از Prompt Command
- برنامه ای با قابلیت اجرای Sequential پشت سر هم دستورات و کد ها
- دریافت ورودی ها و رویدادها تنها با استفاده از صفحه کلید
- عدم پشتیبانی از تمامی Page Code های موجود
- هدف از ایجاد:
- به منظور ایجاد برنامه هایی که نیاز به ظاهر گرافیکی خاصی ندارند و تبادل رویداد های خیلی ساده ای با کاربر دارد
- به منظور ایجاد برنامه هایی برای تست های اتوماتیک که نیاز به منابع خاص گرافیکی و ... نداشته باشد

Basic C# Syntax

- Like c++ and Java
- Ignore White Space
- End with ;
- Block structure
- Indentation for more readability
- C# is case-sensitive

Examples of statements and blocks

using System; // a semicolon indicates the end of a statement

namespace Basics

{ // an open brace indicates the start of a block class Program

{

static void Main(string[] args)

{

 Console.WriteLine("Hello World!"); // a statement }

}

} // a close brace indicates the end of a block

آشنایی با Using

هنگامی که شما از Using استفاده میکنید بدین معنی است که شما میتوانید از کلاسهای اضافه شده به وسیله Using میتوانید استفاده نمایید.

فضای نام یا namespace چیست؟

فضای نام (Namespace) یک مجموعه نماد می باشد که از آن برای سازمان دادن به انواع متفاوت شیء استفاده می شود، هدف آن است که این اشیا بتوانند توسط نام ارجاع شوند. یک فضای نام باید تضمین کند که همه شناسه های موجود در آن نام یکتایی دارند.

Like an address for a type.

Some method in Console Project

- Write
- WriteLine
- ReadKey
- ReadLine
- SetBufferSize
- SetCursorPosition
- ForegroundColor
- BackgroundColor
- Title

متغیر چیست؟

متغیر در برنامه نویسی یک قطعه **نام‌گذاری** شده در **حافظه** کامپیوتر است که **داده‌ها** در آن قابل ذخیره‌سازی هستند. متغیرها در برنامه نویسی را می‌توان مانند جعبه‌ای با یک نام در نظر گرفت که در داخل آن اطلاعاتی ذخیره می‌شوند. به هر اندازه‌ای که در کدهای برنامه نیاز باشد، می‌توان متغیرها را ایجاد، ویرایش و حذف کرد.

متغیرها در برنامه نویسی معمولاً دارای یک نوع داده (Data Type) خاص هستند. که نمایش منطقی و اندازه (Size) آن‌ها را مشخص می‌کند.

لطفاً ابتدا پروژه "**VariableInCSharp**" را مشاهده نمایید.

Naming things and assigning values

Page66 ref

TypeSafe Programming Language

A type-safe programming language is a language that enforces strict type checking during compilation to prevent certain types of programming errors. Type safety ensures that the types of data used in a program are used appropriately and consistently, reducing the likelihood of bugs and improving the reliability and stability of the software.

Features:

- Reduced Errors
- Improved Code Quality
- Increased Security
- Better Maintainability

What makes C# a Strongly Typed Language?

- Type Everything
- Rich .NET Class Library
- Type information (Metadata)
- Compiler Type Safety
- Runtime Type Safety

Predefined Data Types in C# - Part 1

byte	8-bit unsigned integer	0 to 255	
sbyte	8-bit signed integer	-128 to 127	
short	16-bit signed integer	-32,768 to 32,767	
ushort	16-bit unsigned integer	0 to 65,535	
int	32-bit signed integer	-2,147,483,648 to 2,147,483,647	
uint	32-bit unsigned integer	0 to 4,294,967,295	u
long	64-bit signed integer	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	l
ulong	64-bit unsigned integer	0 to 18,446,744,073,709,551,615	ul
float	32-bit Single-precision floating point type	-3.402823e38 to 3.402823e38	f

Predefined Data Types in C# - Part 2

double	64-bit double-precision floating point type	-1.79769313486232e308 to 1.79769313486232e308	d
decimal	128-bit decimal type for financial and monetary calculations	(+ or -)1.0 x 10e-28 to 7.9 x 10e28	m
char	16-bit single Unicode character	Any valid character, e.g. a,*, \x0058 (hex), or\u0058 (Unicode)	
bool	8-bit logical true/false value	True or False	
object	Base type of all other types.		
string	A sequence of Unicode characters		
DateTime	Represents date and time	0:00:00am 1/1/01 to 11:59:59pm 12/31/9999	

Types

- Nouns are types, variables, fields, and properties
- Verbs are methods
- It's worth taking note that the term type is often confused with class.

In C#, every type can be categorized as a class, struct, enum, interface, or delegate. You will learn what these mean in Chapter 6, Implementing Interfaces and Inheriting Classes. As examples, the C# keyword `string` is a class, but `int` is a struct. So, it is best to use the term type to refer to both.

Operator(عملگر) and Operand(عملوند)

5 + 10

5 و 10 Operand هستند و + یک عملگر است.

عملگر (Operator) در برنامه‌نویسی به ساختاری گفته می‌شود که رفتاری مشابه با توابع دارد اما از نظر نحوی یا معنایی با آن‌ها متفاوت می‌باشد. عملگرهای حسابی (از جمله +) و عملگرهای مقایسه‌ای (مثلاً >) نمونه‌هایی از انواع عملگرها هستند که در اغلب زبان‌های برنامه‌نویسی پشتیبانی می‌شود.

Statement vs Expression

In English, we indicate the end of a sentence with a full stop. A sentence can be composed of multiple words and phrases, with the order of words being part of the grammar. For example, in English, we say “the black cat.”

The adjective, black, comes before the noun, cat. Whereas French grammar has a different order; the adjective comes after the noun: “le chat noir.” What’s important to take away from this is that the order matters.

C# indicates the end of a statement with a semicolon. A statement can be composed of multiple variables and expressions. For example, in the following statement, `totalPrice` is a variable and `subtotal + salesTax` is an expression:

```
var totalPrice = subtotal + salesTax;
```

انواع حافظه:

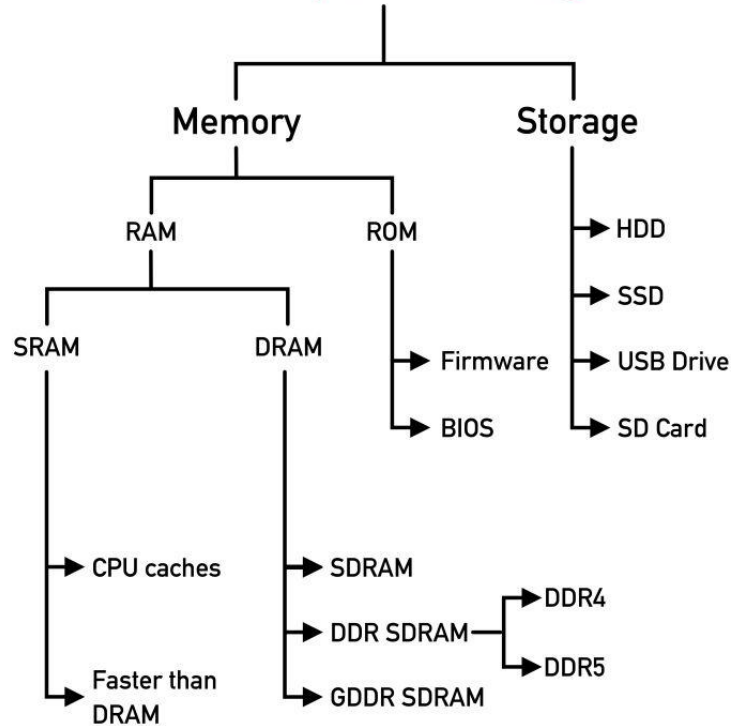
- SAM
- ROM
- RAM (در زمینه نگهداری متغیر این نوع حافظه برای ما اهمیت دارد.)
- PROM

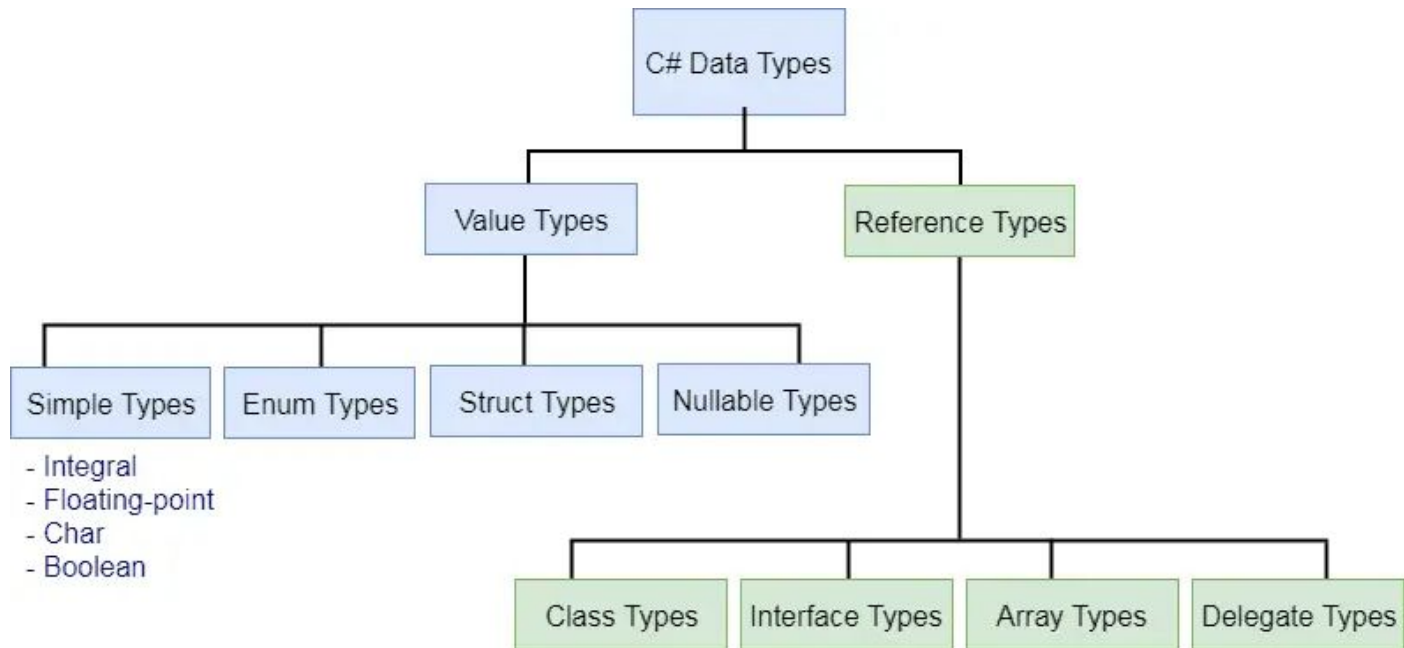
حافظه

- Heap
- Stack

Stack and heap are portions of the memory. The ***Common Language Runtime (CLR)*** allocates memory for objects in these parts.

Types of Memory and Storage





حافظه Stack

- در بخش user-space حافظه قرار دارد.
- به صورت خودکار توسط CPU مدیریت می شود.
- اندازه حافظه stack ثابت است به همین دلیل به آن static memory گفته می شود.
- اطلاعات پشت سر هم و به ترتیب قرار می گیرند
- الگوریتم LIFO (Last In First Out)

سوال: چه نوع متغیرهایی در این حافظه نگهداری می شوند؟

در این حافظه متغیرهایی با نوع داده **ValueType** نگهداری میشوند.

نکته: پارامترها و اطلاعات مربوط به توابع برای اجرا و کنترل آنها نیز در این حافظه ذخیره می شوند.

حافظه Heap

- حافظه Heap در قسمت user-space حافظه مجازی قرار دارد.
- به صورت دستی توسط برنامه نویس مدیریت می شود
- Heap مربوط به زمان اجرا (runtime) است و فضای اشغال شده در heap با اتمام کار تابع آزاد نمی شوند و تا زمانی که Garbage Collector این فضا را آزاد کند یا توسط برنامه نویس داده ها از حافظه heap پاک نشوند در این فضا باقی می ماند. اندازه حافظه heap متغیر است به همین دلیل به آن dynamic memory گفته می شود.
- این حافظه برای نگهداری متغیرهایی با نوع داده **Reference Type** میباشد.

Value Type

A data type is a value type if it **holds a data** value within its **own memory space**. It means the variables of these data types directly contain values.

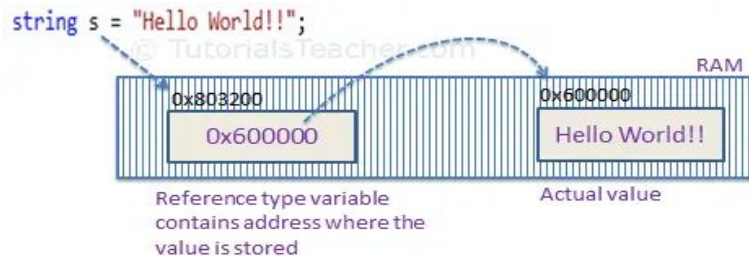


bool
char
double
float
int
long
sbyte
short
struct
uint
ulong
ushort

byte
decimal
enum

Reference Type

Reference type **doesn't store its value directly**. Instead, it **stores the address** where the **value is being stored**. In other words, a reference type contains a **pointer to another memory location** that holds the data



Reference type data types:

- String
- Array
- Class
- Delegate

لطفا ابتدا پروژه "ValueTypeVsReferenceType.4" را مشاهده نمایید.

byte	System.Byte	struct
sbyte	System.SByte	struct
int	System.Int32	struct
uint	System.UInt32	struct
short	System.Int16	struct
ushort	System.UInt16	struct
long	System.Int64	struct
ulong	System.UInt64	struct
float	System.Single	struct
double	System.Double	struct
char	System.Char	struct
bool	System.Boolean	struct
object	System.Object	Class
string	System.String	Class
decimal	System.Decimal	struct
DateTime	System.DateTime	struct

Alias vs .NET Type

```
int i = 345;
```

```
Int32 i = 345; // same as above
```

Default Values

- هر نوع داده یک مقدار اولیه دارد.

- Numeric Types -> 0
- Boolean Types -> false
- Char -> '\0'

برای واضح مطرح کردن مقدار اولیه میتوانید از کلیدواژه `default` استفاده نمایید.

```
int i = default;  
int i = default(int);
```

value types, which means that they must have a value. You can determine the default value of a type by using the `default()` operator and passing the type as a parameter. You can assign the default value of a type by using the `default` keyword.

The string type is a reference type. This means that string variables contain the memory address of a value, not the value itself. A reference type variable can have a null value, which is a literal that indicates that the variable does not reference anything (yet). `null` is the default for all reference types.

Var

در سی شارپ متغیر باید با نوع داده تعریف شود.

حال دو نوع مشخص کردن نوع داده داریم:

- `int i = 100; // explicitly typed variable`
 - `var j = 100; // implicitly typed local variable`
- Explicitly Typed Variable
 - Implicitly Typed Local Variable

مثال:

```
int i = 10;  
var j = i + 1; // compiles as int
```

نکات:

- در هنگام تعریف متغیری با نوع `var` باید مقدار اولیه برای آن در نظر گرفته شود.
- از `var` به عنوان نوع داده ورودی تابع نمی‌توان استفاده کرد.

لطفا پروژه "ImplicitlyTypedVariable.5" را مشاهده نمایید.

Inferring the type of a local variable

You can use the `var` keyword to declare local variables with C# 3 and later. The compiler will infer the type from the value that you assign after the assignment operator, `=`.

A literal number without a decimal point is inferred as an `int` variable, that is, unless you add a suffix, as described in the following list:

- `L`: Compiler infers `long`
- `UL`: Compiler infers `ulong`
- `M`: Compiler infers `decimal`
- `D`: Compiler infers `double`
- `F`: Compiler infers `float`

A literal number with a decimal point is inferred as `double` unless you add the `M` suffix, in which case the compiler infers a decimal variable, or the `F` suffix, in which case it infers a float variable.

Type Conversion

:Type Conversion

- **Implicit conversion:** در این روش کامپایلر تبدیل را به صورت امن انجام میدهد و مناسب برای تبدیل از نوع داده‌ای از ظرف کوچکتر به بزرگتر میباشد. به این روش تبدیل **ضمنی** هم گفته میشود.
- **Explicit conversion:** تبدیل از یک ظرف بزرگ به کوچکتر میباشد که کامپایلر به شما گوشزد میکند که ممکن است data lost اتفاق افتد. به این روش **casting** یا تبدیل **صریح** نیز گفته میشود.
- **Non-compatible Type conversion:** تبدیل از نوع غیر سازگار.

Implicit Type Conversion

```
byte b = 1;                                00000001  
int i = b;    00000000 00000000 00000000 00000001
```

Explicit Type Conversion

```
int i = 1;  
byte b = i;    // won't compile
```

تبدیل نوع غیر سازگار

خیلی وقتا شده که یه عدد رو در قالب رشته در نظر می گیریم و میخوایم اون رو به نوع عددی خودش تبدیل کنیم ، با اینکه میدونیم اون یه عدد اما در حقیقت به عنوان یک رشته به کامپایلر معرفی شده و کامپایلر ما اون رو به عنوان یه نوع رشته ای میشناسه . تو این جور مواقع که میخوایم رشته ای رو به عددی تبدیل کنیم چون نوع هاشون با هم سازگار نیستن نمیتونیم از روش Casting استفاده کنیم . خب پس به یه مکانیزم جدید نیاز داریم .

Non-compatible types

```
string s = "1";  
  
int i = Convert.ToInt32(s);  
  
int j = int.Parse(s);
```

به پروژه "TypeConversionInCSharp.5" مراجعه فرمایید.

TryParse methods:

To avoid errors, you can use the TryParse method instead. TryParse attempts to convert the input string and returns true if it can convert it and false if it cannot. Exceptions are a relatively expensive operation so they should be avoided when you can.

Arithmetic Operators

Operator	Name	Description	Example
+	Addition	Computes the sum of left and right operands.	<code>int x = 5 + 5;</code>
-	Subtraction	Subtract the right operand from the left operand	<code>int x = 5 - 1;</code>
*	Multiplication	Multiply left and right operand	<code>int x = 5 * 1;</code>
/	Division	Divides the left operand by the right operand	<code>int x = 10 / 2;</code>
%	Remainder	Computes the remainder after dividing its left operand by its right operand	<code>int x = 5 % 2;</code>
++	Unary increment	Unary increment ++ operator increases its operand by 1	<code>x++</code>
--	Unary decrement	Unary decrement -- operator decreases its operand by 1	<code>x--</code>
+	Unary plus	Returns the value of operand	<code>+5</code>
-	Unary minus	Computes the numeric negation of its operand.	<code>-5</code>

Assignment Operators

Operator	Name	Description	Example
=	Assignment	Assigns its right hand value to its left-hand variable, property or indexer.	x = 10;
x op= y	Compound assignment	Short form of x =x op y where op = any arithmetic, Boolean logical, and bitwise operator.	x += 5;
??=	Null-coalescing assignment	C# 8 onwards, ??= assigns value of the right operand only if the left operand is null	x ??= 5;

Equality Operators

Operator	Description	Example	
==	Returns true if operands are equal otherwise false.	x == y;	Try it
!=	Returns true if operands are not equal otherwise false.	x != y;	Try it

Catch errors

Comparison Operators

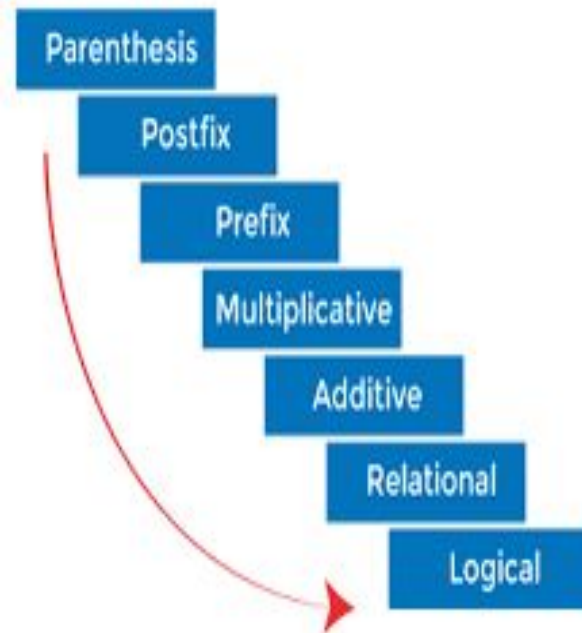
Operator	Description	Example	
==	Returns true if operands are equal otherwise false.	x == y;	Try it
!=	Returns true if operands are not equal otherwise false.	x != y;	Try it

Boolean Logical Operators

Operator	Description	Example	
!	Reverses the bool result of bool expression. Returns false if result is true and returns true if result is false.	!false	Try it
&&	Computes the logical AND of its bool operands. Returns true both operands are true, otherwise returns false.	x && y;	Try it
	Computes the logical OR of its bool operands. Returns true when any one operand is true.	x y;	Try it

تقدم و تاخر عملگر ها (Operator Precedence)

PRECEDENCE	OPERATORS
Highest	++, -- (used as prefixes); +, - (unary)
	*, /, %
	+, -
	==, *==, /=, %==, +=, -=
Lowest	++, -- (used as suffixes)



انواع نام گذاری:

- نام گذاری **PascalCase**: در این شیوه نام گذاری حرف اول همه کلمات با حروف بزرگ نوشته می شود و بین هر کلمه از هیچ کاراکتری (space ، - ، _ و ...) استفاده نمی شود. کاربرد در نامگذاری کلاس ها و property ها.
 - مانند: ClientTarget, UserStatus
- نام گذاری **camelCase**: در این شیوه نام گذاری هر کلمه با حرف کوچک و سایر کلمات با حروف بزرگ نوشته می شود و بین هر کلمه از هیچ کاراکتری (space ، - ، _ و ...) استفاده نمی شود. کاربرد در نامگذاری پارامترهای ارسالی به تابع، متغیر ها و کلیدها در json.
 - مانند: userId, deliveredAtTime
- نام گذاری **snake_case**: در این شیوه نام گذاری هر کلمه با _ (underline) از کلمه قبلی جدا می شود.
 - مانند: user_id, received_at
- نام گذاری **kebab-case**: در این روش نام گذاری هر کلمه با - (dash) از کلمه قبلی جدا می شود. اسامی دیگه این سبک lisp-case, spinal-case, Train-Case. این نامگذاری تقریباً منسوخ شده است و برای query string در url مورد استفاده قرار می گیرد. سایر کاربرد های آن با نامگذاری snake case انجام می شود.
 - مانند: lazy-load, fox-name

Object Name	Notation	Plural
Namespace name	PascalCase	Yes
Class name	PascalCase	No
Constructor name	PascalCase	No
Method name	PascalCase	Yes
Method arguments	camelCase	Yes
Local variables	camelCase	Yes
Constants name	PascalCase	No
Field name Public	PascalCase	Yes
Field name Private	_camelCase	Yes
Properties name	PascalCase	Yes
Interface	IPascalCase	No
Enum type name	PascalCase	Yes

Adnan Maqbool Khan

Summarizing options for storing text

Literal string: Characters enclosed in double-quote characters. They can use escape characters like `\t` for tab. To represent a backslash, use two: `\\`.

Raw string literal: Characters enclosed in three or more double-quote characters.

Verbatim string: A literal string prefixed with `@` to disable escape characters so that a backslash is a backslash. It also allows the string value to span multiple lines because the whitespace characters are treated as themselves instead of instructions to the compiler.

Interpolated string: A literal string prefixed with `$` to enable embedded formatted variables. You will learn more about this later in this chapter.

Importing a static type for all code files in a project

```
<ItemGroup>
```

```
  <Using Include="System.Console" Static="true" />
```

```
</ItemGroup>
```

Miscellaneous operators

sizeof and sizeof are convenient operators when working with types:

- sizeof returns the short name (without the namespace) of a variable, type, or member as a string value, which is useful when outputting exception messages.
- sizeof returns the size in bytes of simple types, which is useful for determining the efficiency of data storage.



Setting a breakpoint and starting debugging

Visual Studio 2022

Standard Toolbar

Debug Toolbar



↑
Continue

↑
Hot Reload

↑
Break All

↑
Stop Debugging

↑
Restart

↑
Show Next Statement

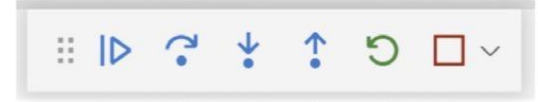
↑
Step Into

↑
Step Over

↑
Step Out

↑
Show Threads in Source

Visual Studio Code



↑
Continue

↑
Step Over

↑
Step Into

↑
Step Out

↑
Restart

↑
Stop

Writing Method

A Fundamental principle of programming is **Don't Repeat Yourself(DRY)**.

Function are like tiny programs that complete one small task.

A method is a block of code which only runs when it is called.

You can pass data, known as **parameters**, into a method.

Documenting functions with XML comments

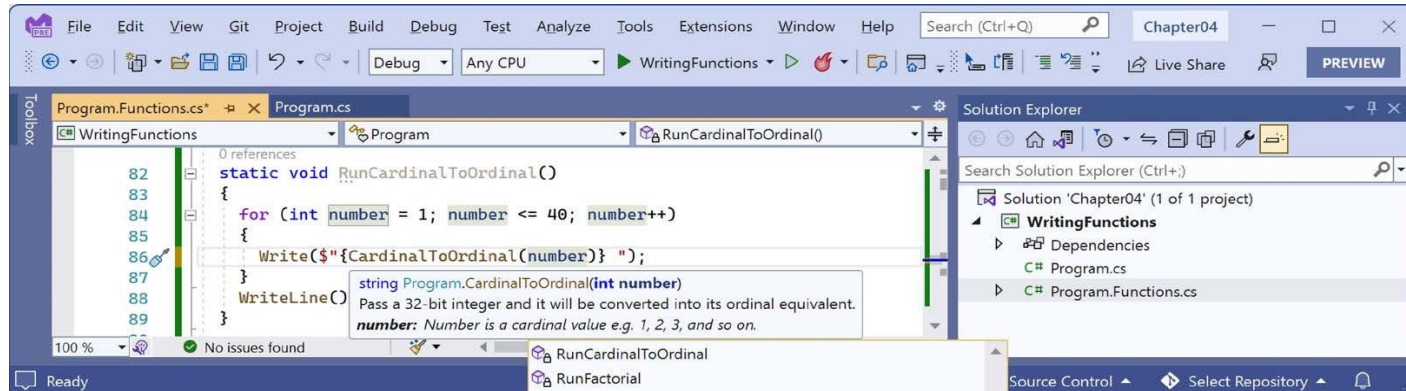
```
/// <summary>
```

```
/// Pass a 32-bit integer and it will be converted into its ordinal equivalent.
```

```
/// </summary>
```

```
/// <param name="number">Number as a cardinal value e.g. 1, 2, 3, and so on.</param>
```

```
/// <returns>Number as an ordinal value e.g. 1st, 2nd, 3rd, and so on.</ returns>
```



شاخه بندی کد: (Branching)

انواع:

- Ternary statement
- If statement
- Switch statement

حلقه‌های تکرار:

- Do loops
- While Loops
- For Loops

ایجاد وقفه در حلقه:

- Break
- Continue
- Goto
- Return

حلقه‌های بی‌نهایت

For Loop

```
for (statement 1; statement 2; statement 3)

{

    // code block to be executed

}
```

Statement 1 is executed (one time) before the execution of the code block.

Statement 2 defines the condition for executing the code block.

Statement 3 is executed (every time) after the code block has been executed.

While

```
while (condition)

{

    // code block to be executed

}
```

در این نوع حلقه باید حواسمان باشد که شرط را مدیریت کنیم.

break

ما از این مفهوم برای jump out کردن استفاده میکنیم.

What is the difference between break, continue, and return when used inside a loop statement?

What are the three parts of a for statement and which of them are required?

Converting from any type to a string

The most common conversion is from any type into a string variable for outputting as human-readable text, so all types have a method named `ToString` that they inherit from the `System.Object` class. The `ToString` method converts the current value of any variable into a textual representation. Some types can't be sensibly represented as text, so they return their namespace and type name instead.

Building your own types with OOP

you will learn about all the different categories of members that a type can have, including fields to store data and methods to perform actions. You will use OOP concepts such as aggregation and encapsulation.

You will also learn about language features such as tuple syntax support, out variables, inferred tuple names, and default literals. You will learn about defining operators and local functions for performing simple actions

[Go to OOP Slider:.](#)

OOP

An object in the real world is a **thing**, such as a car or a person, whereas an object in programming often represents something in the real world, such as a product or bank account, but it can also be something more abstract.

Concepts of OOP:

- Encapsulation
- Composition
- Aggregation
- Inheritance
- Abstraction
- Polymorphism